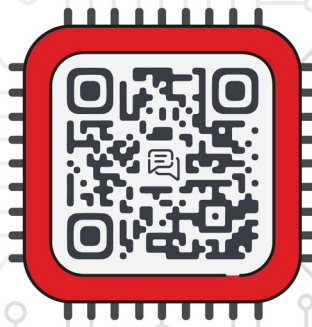




نظري

4

12

1440 sp

كلية الهندسة المعلوماتية

السنة الرابعة - النظم والشبكات

Non-Blocking I/O

د. غسان شحود



RB Informatcs; 27/11/2025

برمجة تطبيقات شبكية

السلام عليكم ورحمة الله وبركاته

في البرمجة (networking) تكون غالباً النقطة التي تبطل العملية الكلية هي الشبكة، وذلك بسبب وجود سرعة نقل محدودة والتأخير الحاصل بسبب انتظار أن تصل البيانات من الـ client والقيود العملية المفروضة. إذاً فإن التأخير يأتي من الـ CPU، وغالباً يكون من الشبكة.

هناك طريقتان لإدخال وإخراج البيانات في الشبكة:

1. Blocking I/O :

- 1- كل اتصال يحتاج لـ Thread.
- 2- عند القراءة والكتابة على الشبكة: يتوقف عمل الـ Thread وينتظر حتى تصل البيانات أو يتم إرسالها.
- 3- وهذا يؤدي إلى استهلاك ذاكرة كبير وتكلفة عالية في التبديل بين الـ Threads (Context Switching).
- 4- أداء ضعيف عندما يكون لدينا آلاف الاتصالات

2. Non-Blocking I/O :

- 1- لحل المشكلة التي تسببها الـ Blocking I/O.
- 2- بدلاً من أن كل اتصال له Thread خاص به، أصبح الـ Thread الواحد يدير عدد كبير من الاتصالات.
- 3- مبدأ عمله: الـ Thread لا ينتظر الشبكة، فهو يرى إذا كان هناك اتصال جاهز للقراءة أو الكتابة، فيكتب ويقرأ أكبر كمية ممكنة فوراً ثم ينتقل مباشرة إلى اتصال آخر.
- 4- أي: لا يوجد انتظار ولا تجميد للـ Thread، ويتم استغلال الـ CPU بشكل ممتاز.

Java NIO (New I/O)

وفرت جافا أسلوب الـ Non-Blocking في (java.nio) packet وتشمل أهم المكونات:

المكونات الأساسية:

1- Selectors :

- 1- بدون الـ Non-Blocking هناك.
- 2- يسمح لـ Thread واحد بمراقبة آلاف القنوات.
- 3- أي قناة جاهزة للقراءة أو الكتابة أو لقبول اتصال جديد.

2- Buffers :

- 1- مكان مؤقت لتخزين البيانات أثناء القراءة والكتابة.
- 2- مثل: Byte Buffer, Char Buffer.

3- Channels :

- 1- بديل الـ Streams وأكثر كفاءة منه.
- 2- يمكن أن تكون Blocking أو Non-Blocking.

لماذا Non-Blocking I/O؟

- 1- يسمح للـ CPU بالعمل باستمرار بدون انتظار الشبكة.
- 2- يقلل عدد الـ Threads بشكل كبير، ويعتمد على:
 - I. java.nio
 - II. channels
 - III. Buffers
 - IV. selectors
- 3- مهم جداً لاستخدامه للسيرفرات ذات الأداء العالي (High-Performance Servers).

لنفهم المشكلة أكثر:

في الـ Blocking كل client يتصل بالسيرفر يتم إنشاء Thread خاص به، فإن هذا الـ Thread ينتظر بيانات من الشبكة (ينتظر حتى يرسل البيانات)، لكن المشكلة أن الـ Thread يكون موجود في الذاكرة لكنه فعلياً لا يعمل، ومجرد انتظار للشبكة، حيث أن عمليات Socket I/O أبطأ بكثير من عمليات الـ CPU، حيث أن المعالج يستطيع تنفيذ ملايين العمليات في الثانية، بينما الشبكة قد تحتاج وقت أكثر.

The C10k problem

وهي تعني القدرة على التعامل مع 10,000 اتصال متزامن أو أكثر على سيرفر واحد.

في حالة الـ Blocking: نحتاج لـ 10,000 Thread (Thread per connection).
 عند 10,000 اتصال يعني 10,000 Threads، وكل Thread يحتاج تقريباً 1MB، يعني 10GB ذاكرة تقريباً.
 ويضيع وقت المعالج في الـ Context Switching.
 وهذا يؤدي إلى استهلاك ذاكرة ضخم وبطء شديد وانهايار السيرفر أحياناً.

Blocking I/O	Non-Blocking I/O
ينتظر البيانات	لا ينتظر وينتقل للاتصالات أخرى
Thread لكل اتصال	Thread واحد لعدة اتصالات
استهلاك موارد كبير	استهلاك موارد أقل
سهل	أعقد لكن أقوى
لا يناسب الـ C10k	حل مثالي للـ C10k

Features in Java I/O

تم تقديم Java NIO (New I/O) لتقديم:

1. أداء أعلى.
 2. قابلية توسع أفضل.
 3. دعم Non-Blocking I/O.
- وتعتمد بشكل أساسي على: Selectors, Channels, Buffers.

Selector ↩

1. هو أهم جزء، بدونه سنحتاج لـ Thread لكل اتصال.
2. يجعل Thread واحد يراقب آلاف الاتصالات، لا ينتظر ولا يستهلك موارد دون فائدة.
3. هو أساس الـ Non-Blocking I/O.

Channels ↩

1. الاتصال الفعلي عبر الدخل والخرج.
2. الفرق بين الـ Stream والـ Channel:

Stream	Channel
اتجاه واحد	اتجاهين (قراءة وكتابة)
Blocking فقط	Blocking و Non-Blocking
لا يعمل مع Selector	يعمل مع Selector

Buffers

هو حاوية مؤقتة يجب أن تمر عبره كل البيانات لأنه لا يمكن الإرسال مباشرة إلى الـ Channel أو القراءة منه، لذلك كل شيء يتم عبر الـ Buffer حيث يتم معالجة البيانات داخله.

الـ Packages الموجودة في Java NIO:

1. java.nio: تحتوي على الـ Buffers والأدوات الأساسية للتعامل مع الذاكرة، مثل Byte Buffer و Char Buffer.
2. java.nio.channels: تحتوي على الـ Channels و الـ Selectors، مثل:
 - SocketChannel-1
 - ServerSocketChannel-2
 - FileChannel-3
 - Selector-4
3. java.nio.charset: مسؤولة عن ترميز الأحرف للتحويل من Bytes إلى characters، مثل: UTF-8 و UTF-16.

Buffers

هو نقطة دخول وخروج البيانات، فهو object يخزن البيانات مؤقتاً، إما بيانات تمت قراءتها أو بيانات سيتم كتابتها (من وإلى) الـ Channels.

كيف يعمل الـ Buffer في NIO؟

- عند القراءة: يقرأ الـ Channel البيانات ثم توضع داخل الـ Buffer، ثم يقوم البرنامج بقراءة البيانات من الـ Buffer.
- عند الكتابة: البرنامج يضع البيانات في الـ Buffer، ثم يأخذ الـ Channel البيانات من الـ Buffer، ثم يتم إرسال البيانات.

أنواعها:

ByteBuffer
CharBuffer
IntBuffer
LongBuffer
FloatBuffer
DoubleBuffer

تقنياً: الـ Buffer هو مصفوفة Bytes لكن عملياً: هو أكثر من مجرد مصفوفة لأنه ينظم الوصول للبيانات عن طريق تتبعها.

الخصائص الأساسية في الـ Buffer:

1. Capacity (السعة): تمثل الحجم الكلي للـ Buffer، يتم تحديده عند إنشاء الـ Buffer ولا يمكن تغييره.
`ByteBuffer buffer = ByteBuffer.allocate(1024);`
2. Position (الموضع): يحدد أين ستتم عملية (القراءة، الكتابة) التالية. ويتغير تلقائياً بعد كل عملية get أو put :
`Postion = postion +1`

Limit.3: يحدد نهاية البيانات الصالحة للقراءة أو الكتابة، ولا يمكن القراءة أو الكتابة بعد هذا الحد.

Limit <= Capacity

Mark.4: يستخدم في بعض المواضيع المتقدمة، وهو علامة اختيارية يمكن الرجوع إليها باستخدام reset().

العلاقة بينهم: zero(0) <= mark <= position <= Limit <= capacity

وهذه العلاقة يتم ضمان عدم تجاوز حدود الذاكرة وسلامة القراءة والكتابة.

حالات ال Buffer

1. Writing (الكتابة)

عندما نريد القراءة من ال Channel أو وضع بيانات جديدة.

في هذه الحالة نستخدم clear() حيث تجعل ال Buffer جاهز لاستقبال بيانات جديدة، بحيث:

position = 0

Limit = capacity

2. Reading (القراءة)

عندما نريد إرسال بيانات عبر ال Channel أو قراءتها من البرنامج.

في هذه الحالة نستخدم flip() حيث تجعل البيانات جاهزة للقراءة من البداية وحتى آخر البيانات المكتوبة، وذلك بجعل:

Limit = position

position = 0

أهم كلاسات ال Buffer

1. Buffer (Super class): وهو الكلاس الأب لكل أنواع ال Buffer، ويحتوي على الخصائص المشتركة.

2. ByteBuffer: وهو الأكثر استخداماً لتعريف ال Buffer ويدعم دوال ال allocate(), put(), get().

مثال: ByteBuffer buffer = ByteBuffer.allocate(1024);

أهم ال Methods في ال Buffer

allocate(int capacity): إنشاء Buffer بسعة محددة.

clear(): تهيئة ال Buffer للكتابة، تجعل position = 0 و Limit = capacity.

flip(): تهيئة ال Buffer للقراءة، تجعل Limit = position و position = 0.

get(): قراءة عنصر من ال Buffer.

put(): كتابة عنصر في ال Buffer.

(في كل عملية يتم زيادة ال position بمقدار واحد)

hasRemaining(): للتحقق إذا كان هناك بيانات غير مقروءة، أي: position < Limit. ترجع true أو false.

Channels

الـ Channel هو object يمثل اتصال مفتوح يسمح بعمليات القراءة والكتابة، لذلك هو ثنائي الاتجاه (Bi-directional). هي الطريق التي تمر عبره البيانات لكنه لا يخزنها، بل ينقل البيانات من وإلى الـ Buffer.

- عند القراءة: الـ Channel يقرأ البيانات من المصدر (ملف، شبكة، ...) ويضع هذه البيانات داخل Buffer.

```
;channel.read(buffer)
```

- عند الكتابة: يأخذ الـ Channel البيانات من الـ Buffer ويرسلها إلى الوجهة.

```
;channel.write(buffer)
```

أنواع الـ Channels

1. ServerSocketChannel: مخصص لاستقبال الاتصالات الجديدة (Accept connections). يستخدم في السيرفرات.

```
;()ServerSocketChannel ssc = ServerSocketChannel.open
```

2. SocketChannel: مخصص للتواصل عبر TCP، ويستخدم بعد قبول الاتصال.

3. DatagramChannel: مخصص للـ UDP connection، غير معتمد على اتصال دائم.

4. FileChannel: مخصص للقراءة والكتابة من الملفات، لا يعمل مع Selector لكنه جزء من NIO.

أهم الـ Methods في الـ Channels

1- open(): تستخدم لفتح channel جديدة :

```
SocketChannel.open() , ServerSocketChannel.open()
```

2- bind(SocketAddress local): يربط الـ channel بعنوان محلي (IP + Port)، ويستخدم عادة مع ServerSocketChannel.

```
;ssc.bind(new InetSocketAddress(8000))
```

3- accept(): تستخدم مع ServerSocketChannel لقبول اتصال جديد من عميل، وترجع SocketChannel:

```
SocketChannel client = ssc.accept()
```

وفي Non-Blocking Mode ترجع null بدلاً من الانتظار إذا لم يوجد اتصال.

4- read(ByteBuffer dst): قراءة البيانات من الـ Channel وتخزينها في الـ Buffer:

```
Channel.read(buffer);
```

ترجع عدد البيانات المقروءة أو -1 إذا انتهى الاتصال.

5- write(ByteBuffer src): كتابة البيانات من الـ Buffer إلى الـ Channel :

```
Channel.write(buffer);
```

6- socket(): تستخدم للحصول على ال Socket المرتبط بال Channel.

7- connect(SocketAddress remote): ربط SocketChannel بعنوان (سيرفر).

```
;channel.connect(new InetSocketAddress("localhost", 8000))
```

8- configureBlocking(boolean blocking): تغيير وضع ال Blocking -> true, Non-Blocking -> false.

```
;channel.configureBlocking(false)
```

لنشرح كيف يتم إنشاء ServerSocketChannel:

يتم إنشاؤها في Java NIO لاستقبال اتصالات العملاء (clients) والعمل مع selector.

Code:

```
1. ServerSocketChannel ssc = ServerSocketChannel.open();
2. ssc.configureBlocking(false);
3. ServerSocket ss = ssc.socket();
4. SocketAddress address = new InetSocketAddress(80);
5. ss.bind(address);
```

شرح الخطوات:

1. ننشئ كائن ServerSocketChannel هو فقط لفتح Socket مخصص للشبكة.
2. يحول القناة من Blocking Mode إلى Non-Blocking Mode حتى نستطيع العمل مع ال selector.
3. الحصول على ال ServerSocket المرتبط بال channel.
4. إنشاء عنوان الشبكة، وهنا يأخذ الشبكة البورت 80.
5. ربط السيرفر بالعنوان، حيث يتم ربط ال ServerSocket بالمنفذ المحدد وفتح المنفذ فعلياً على النظام، وهنا يصبح السيرفر جاهز لاستقبال الاتصالات.

Selectors

ال selector هو central object يراقب عدة channels في نفس الوقت، ويخبرنا أي channel جاهزة للقراءة أو الكتابة أو قبول اتصال جديد و بدون ذلك نحتاج ل Thread لكل اتصال وهدر للموارد، أما مع ال selector يتم إدارة آلاف الاتصالات ب Thread واحد وبدون هدر موارد.

```
Selector selector = Selector.open();
```

تسجيل ال Channel مع ال Selector:

```
;SelectionKey key = ssc.register(selector, SelectionKey.OP_ACCEPT)
```

ssc: هو الـ `ServerSocketChannel`.

selector: هو الـ `selector` الذي سيقوم بالمراقبة.

`SelectionKey.OP_ACCEPT`: نوع الحدث الذي نريد مراقبته (هنا: انتظار اتصال جديد).

`SelectionKey`: هو كائن يمثل العلاقة بين الـ `selector` والـ `channel` ويحتفظ بمعلومات مثل (الأحداث المسجلة، الأحداث الجاهزة حالياً).

Operation Sets ↩

`OP_READ`: (`SocketChannel`) عندما تصل بيانات من العميل.

`OP_WRITE`: (`SocketChannel`) عندما يكون بالإمكان إرسال بيانات.

`OP_ACCEPT`: (`ServerSocketChannel`) عندما يصل client جديد.

`OP_CONNECT`: (`SocketChannel`) أثناء إنشاء اتصال غير متزامن.

تسجيل Selector مع SocketChannel مثلاً: ↩

```
;SocketChannel.register(selector, SelectionKey.OP_READ | SelectionKey.OP_WRITE)
```

أي أننا نطلب من الـ `selector` إخبارنا عندما يكون جاهزاً للقراءة والكتابة.

ملاحظة: أي `channel` يتم تسجيلها مع `selector` يجب أن تكون `Non-Blocking`.

```
;channel.configureBlocking(false)
```

وبدون ذلك سيحدث `Exception` ولن يعمل الـ `selector`.

كيف يعمل الـ SelectionKey في الإشارات؟ ↩

عندما يحدث شيء على الـ `channel` المرتبط بالـ `selector`، الـ `selector` يضع الـ `selectionkey` في قائمة تسمى (`selected keys`) ومن خلالها نعرف الـ `channel` الذي حدث فيه الحدث.

عادة يوجد حلقة `selector.select()` لسؤال الـ `selector` بالاتصال.

Inner Loop (selector Loop)

```
while (true) {
    selector.select(); // Wait for registered events
    Set<SelectionKey> readyKeys =
        selector.selectedKeys();
    Iterator<SelectionKey> it =
        readyKeys.iterator();
    while (it.hasNext()) {
        SelectionKey key = it.next();
        iterator.remove();
        // ... deal with I/O event ...
    }
}
```

انتظار أحداث - معرفة القنوات الجاهزة - معالجتها واحدة بواحدة.

السيرفر يجب أن يبقى يعمل دائماً لذلك نضع حلقة لانهاية حيث ينتظر

الأحداث والأحداث تأتي باستمرار.

قيمة الإرجاع من `select()`:

0 إذا لم يوجد أي حدث.

قيمة موجبة أي أكبر من الصفر >0 إذا وجد حدث أو أكثر.

الدالة `selectedKeys()`:

تعيد هذه الدالة `SelectionKey < Set <` (وهي نوع من `Collections` لا تسمح بوجود عناصر مكررة) تمثل قنوات جاهزة وحدث ما حصل عليها.

باستخدام `Iterator`: لأن الـ `Set` لا يمكن الوصول لعناصرها مباشرة بالـ `index` لذلك يسمح لنا بالمرور على العناصر واحداً واحداً وإزالتها بأمان.

سطر `remove`: مهم لأنه إذا لم نزل المفتاح فإن الـ `selector` لا يفرغ تلقائياً، فإذا لم نزل المفتاح يدوياً، وإلا سيتم معالجته مرة أخرى وتحدث خطأ.

(Accepting Connections)

عندما يحاول عميل جديد الاتصال بالخادم، يجب على الخادم قبول هذا الاتصال وإعداده للعمل.

الخطوات :

1. التحقق: `key.isAcceptable()` يتحقق إذا كان هناك اتصال جديد.
2. الحصول على القناة: `key.channel()` يحصل على القناة التي تستقبل الاتصالات.
3. قبول الاتصال: `server.accept()` يقبل الاتصال الجديد ويخلق قناة جديدة.
4. إعداد القناة: `configureBlocking(false)` يجعل القناة تعمل بشكل غير متزامن.
5. التسجيل: `register(selector, OP_WRITE)` يسجل القنوات الجديدة في `Selector` لمراقبة الأحداث.

ملاحظة مهمة: `NIO` حتى تعمل بشكل صحيح مع (`non-blocking`) يجب دائماً جعل القنوات الجديدة غير متزامنة.

(Removing the processed key)

عندما يتم معالجة حدث، يجب إزالته من القائمة حتى لا يتم معالجته مرة أخرى في الدورة التالية. باستخدام الأمر `it.remove()` بعد معالجة الأحداث.

ماذا يحدث إذا لم نقم بالإزالة؟

إذا لم نقم بإزالة المفتاح، فسيتم معالجته مرة أخرى في الدورة التالية حتى لو لم يحدث عليه حدث جديد، مما يؤدي إلى أخطاء في البرنامج.

(Reading Data)

عندما تكون هناك بيانات متاحة للقراءة، نقوم بقراءتها من القناة إلى `Buffer`.

الخطوات:

1. التحقق: يتحقق إذا كانت هناك بيانات متاحة `key.isReadable()`.
2. إفريغ الـ `Buffer`: لاستقبال بيانات جديدة يفرغ الـ `Buffer` بـ `buffer.clear()`.
3. القراءة: الـ `Buffer` يقرأ البيانات من القناة إلى الـ `cc.read(buffer)`.
4. تحويل الوضع: يحول الوضع من الكتابة إلى القراءة بـ `buffer.flip()`.

5. استخدام البيانات: نستخدم البيانات الموجودة في ال Buffer.

6. إعادة التهيئة: يعيد تهيئة ال Buffer لاستقبال بيانات جديدة ب `buffer.clear()`.

ملاحظة مهمة: يجب استخدام `flip()` بعد القراءة وقبل استخدام البيانات لأن ال بافر يكون في وضع الكتابة بعد القراءة.

(Limitations of Raw NIO)

ما هي القيود؟

NIO قوي ولكن له بعض القيود التي تجعله صعباً في الاستخدام:

1. إدارة الذاكرة المعقدة: يجب على المطور إدارة ال Buffer يدوياً وهو أمر معقد.
2. تتبع حالة الاتصال يدوياً: يجب تتبع حالة كل اتصال يدوياً، مما يزيد من التعقيد.
3. تعقيد تنفيذ البروتوكولات: تنفيذ بروتوكولات مثل HTTP, FTP وهذا يتطلب معرفة وتنفيذ عميق.
4. حالات خطأ محتملة: هناك العديد من الحالات الحدية التي يمكن أن تؤدي إلى أخطاء.

(NIO in the Real World)

هناك العديد من الأنظمة والتطبيقات تستخدم NIO مثل:

- Netty: إطار عمل يبنى على NIO.
- Tomcat NIO: خادم ويب يستخدم NIO.
- Jetty: خادم ويب آخر يستخدم NIO.
- Spring WebFlux: إطار عمل Reactive يستخدم NIO.
- gRPC Java: إطار عمل RPC يستخدم NIO.
- Cassandra: قاعدة بيانات NoSQL تستخدم NIO.
- Kafka: نظام نشر/اشتراك رسائل يستخدم NIO.

لماذا تستخدم هذه الأنظمة NIO؟

1. أداء عالي: يمكن معالجة آلاف الاتصالات في وقت واحد.
2. كفاءة في استخدام الذاكرة: لا حاجة لإنشاء خيط لكل اتصال.
3. قابلية التوسع: يمكن للتطبيقات التوسع لمعالجة أعداد كبيرة من الاتصالات.

Echo-back (Example - Echo-back Server)

```

import java.io.IOException;
import java.net.InetSocketAddress;
import java.nio.ByteBuffer;
import java.nio.channels.*;
import java.util.Iterator;

public class NioEchoServer {
    private Selector selector;

    public void start(int port) throws IOException {
        selector = Selector.open();
        ServerSocketChannel server = ServerSocketChannel.open();
        server.configureBlocking(false).bind(new InetSocketAddress(port))
            .register(selector, SelectionKey.OP_ACCEPT);

        System.out.println("Echo server on port: " + port);
        eventLoop();
    }

    private void eventLoop() throws IOException {
        while (true) {
            selector.select(5000);
            for (Iterator<SelectionKey> it = selector.selectedKeys().iterator(); it.hasNext(); ) {
                SelectionKey key = it.next(); it.remove();
                if (!key.isValid()) continue;

                if (key.isAcceptable()) {
                    SocketChannel client = ((ServerSocketChannel)key.channel()).accept();
                    if (client != null) {
                        client.configureBlocking(false).register(selector, SelectionKey.OP_READ);
                        System.out.println("Client connected: " + client.getRemoteAddress());
                        client.write(ByteBuffer.wrap("welcome to echo server".getBytes()));
                    }
                }

                if (key.isReadable()) {
                    SocketChannel client = (SocketChannel) key.channel();
                    ByteBuffer buf = ByteBuffer.allocate(1024);
                    int bytes = client.read(buf);

                    if (bytes == -1 || "exit".equalsIgnoreCase(new String(buf.array(), 0, bytes).trim())) {
                        System.out.println("Client disconnected: " + client.getRemoteAddress());
                        client.close(); key.cancel(); continue;
                    }

                    if (bytes > 0) {
                        buf.flip(); String msg = new String(buf.array(), 0, buf.limit()).trim();
                        System.out.println "[" + client.getRemoteAddress() + "]: " + msg;
                        client.write(buf);
                    }
                }
            }
        }
    }

    public static void main(String[] args) throws IOException {
        new NioEchoServer().start(8080);
    }
}

```

الفائدة:

هذا المثال يوضح كيف يمكن ل NIO معالجة العديد من الاتصالات في نفس الوقت بكفاءة عالية.

1. خادم Echo بتقنية NIO: يعالج العديد من الاتصالات في وقت واحد بكفاءة عالية
2. ال Selector كمراقب: يتابع جميع القنوات ويحدد الأحداث الجاهزة (قبول أو قراءة)
3. غير محجوز (Non-blocking): لا ينتظر كل عميل، ينتقل بسرعة للعميل التالي
4. وظيفة ال Echo: يعيد إرسال الرسالة نفسها للعميل الذي أرسلها
5. أمر الخروج: كتابة "exit" تغلق الاتصال مع العميل
6. المنفذ 8080: يرسل رسالة ترحيب لكل عميل جديد عند الاتصال

Netty (NIO in the Real World – Netty)

ما هو NETTY :

Netty هو إطار عمل Java يبنى على NIO.

لماذا Netty مهم؟

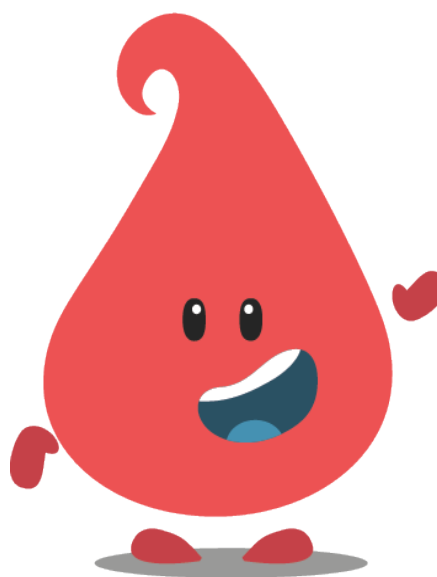
1. يخفي تعقيد NIO: يدير جميع التعقيدات الخفية تلقائياً.
2. واجهة برمجية بسيطة: يوفر واجهة برمجية سهلة الاستخدام.
3. بنية التطبيقات: يمكن بناء نفس التطبيقات بـ 10٪ من الكود مقارنة بـ NIO الخام.
4. أداء عالي: يستخدم NIO بكفاءة عالية.

الفوائد:

- تقليل التعقيد: الكود أبسط وأسهل في الفهم.
- تقليل وقت التطوير: يمكن بناء التطبيقات بسرعة أكبر.
- تقليل الأخطاء: يتعامل مع الحالات الحدية تلقائياً.
- صيانة أسهل: الكود أسهل في الصيانة والتحديث.

Netty أمثلة على استخدام:

1. Spring WebFlux (يستخدم Reactor Netty)
2. gRPC Java
3. Cassandra
4. Minecraft



THE END